
Chapter 3 | GEMM 控制器与卷积数据通路

- 本章目标：让脉动阵列真正计算卷积。
- 内容：SRAM、控制器 FSM、im2col/line buffer、激活/池化/FC。
- 验收：小规模 golden 逐元素一致。

Chapter 3 | 阵列输出的意义

Ch2 的阵列输出是 $C[k][col] = \text{sum_row } A[\text{row}][k] * W[\text{row}][col]$ 。如果我们把 A 的行当作输入通道、列当作空间位置，W 的行当作输入通道、列当作输出通道，那么每个 k（每一拍）就是一个像素点上全部 N 个输出通道。

这个格式非常“顺手”：卷积控制器只要逐像素产生输入向量，收集输出向量，就能完成一层卷积。

回想矩阵乘法：C 的每一格 = A 的一行点乘 B 的一列。我们让 A 的“行”是输入通道，B 的“列”是输出通道，那么 C 的一列就是“一个像素的所有输出通道”。

所以阵列每拍吐出一个向量，正好是卷积输出里一个像素点的完整结果。控制器不需要重新拼装，直接写进输出 SRAM 就行。

- $C[k][col] = \text{sum_row } A[\text{row}][k] * W[\text{row}][col]$ 。
- 把 A 的行当作输入通道、列当作空间位置。
- W 的行是输入通道、列是输出通道。
- 所以一拍算出一个像素的 N 个输出通道。

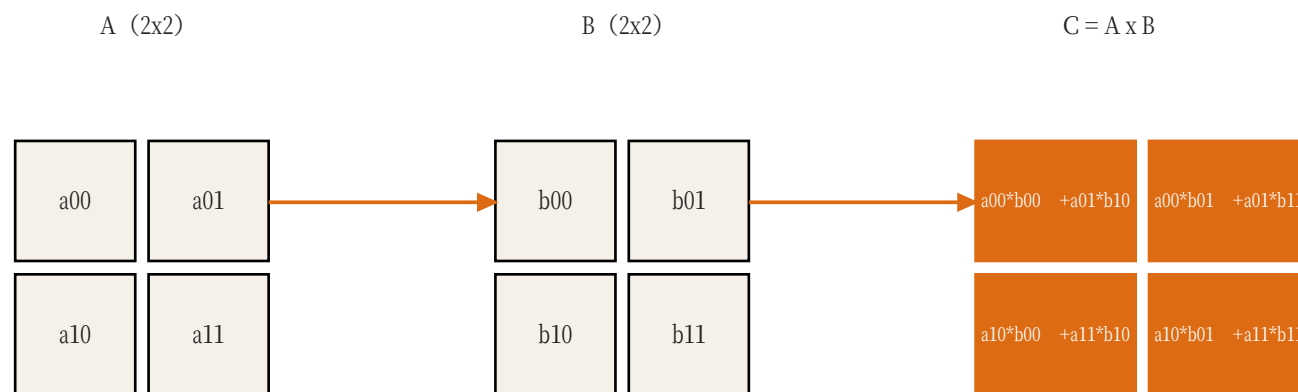


图 C : C 的每个格子 = A 的一行 与 B 的一列 逐项相乘再相加。

Chapter 3 | SRAM 组织

SRAM 是 NPU 的“粮仓”：输入 SRAM 存放特征图，权重 SRAM 存放模型参数，输出 SRAM 暂存结果。它们都比 Hack 的 64KB RAM 更适合阵列并行读取。

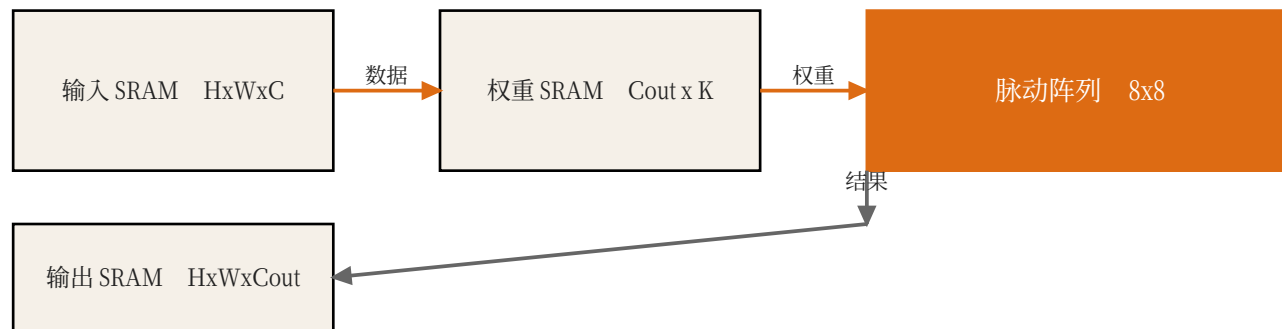
容量估算例子：28x28x8 的 int8 特征图 = 6272 字节；一个小模型约 5 万参数 = 50KB 权重，足够放在片上。

仓库要分区：原料（输入特征图）放 A 区，模具（权重）放 B 区，成品（输出特征图）放 C 区。每个区用地址编号，控制器按编号取货。

容量估算很简单：一个 int8 数占 1 字节，所以 28x28x8 的特征图就是 $28 \times 28 \times 8 = 6272$ 字节。先学会算这个，后面设计 SRAM 才有数。

- 输入 SRAM：H x W x C_{in}，按通道-空间布局。
- 权重 SRAM：C_{out} x C_{in} x 3 x 3，按 im2col 后矩阵布局。
- 输出 SRAM：H x W x C_{out}。
- 规模估算：28x28x8 = 6272 字节，int8。

图 11：SRAM 负责把数据喂给阵列，并接住结果。



Chapter 3 | 控制器 FSM

控制器本质是一个有限状态机：IDLE 等待命令；LOAD_W 把权重搬进阵列；RUN 逐像素送数据、收结果；DRAIN 等流水排空；DONE 通知 CPU。

写控制器时最难的是“地址计算”：每个像素的窗口在输入 SRAM 里对应的偏移、每个输出写回的位置，都要算对。

状态机就像红绿灯：红灯等（IDLE），绿灯放行（RUN），黄灯清空路口（DRAIN）。控制器只做三件事：等命令、按顺序送数据、收结果。

最容易被忽略的是 DRAIN：最后一拍输入后，流水线里还有数据没走完，必须等它排空，否则结果会丢。

- IDLE：等待 CMD。
- LOAD_W：从权重 SRAM 装载阵列。
- RUN：逐像素产生 a_data 并收集 psum_out。
- DRAIN：等待流水排空。
- DONE：置位状态寄存器。



图9：控制器就是一个小型状态机。

Chapter 3 | im2col

im2col 的意思是 “image to column”：把每个卷积窗口展平成一行。一个 3×3 、输入通道为 C 的窗口，展平后长度是 $9 \times C$ ；所有窗口拼起来就是一个大矩阵，矩阵乘的结果就是卷积输出。

代价是输入数据被重复复制（每个像素可能出现在多个窗口里），好处是阵列可以直接做矩阵乘，控制器逻辑简单。

im2col 像把一张照片剪成很多小卡片：每个 3×3

窗口剪成一张长条，所有长条排成一摞，就变成矩阵。矩阵乘完再把结果按位置贴回去。

代价是同一块像素会出现在多张卡片里（重复存储），好处是计算部分变成了纯粹的矩阵乘，控制器非常好写。

- 把 3×3 卷积窗口展开成行向量。
- 每个输出像素对应一个 $K = C_{in} \times 3 \times 3$ 维向量。
- 一行一行喂给阵列，等价于矩阵乘。
- 代价是数据重复，但硬件实现简单。

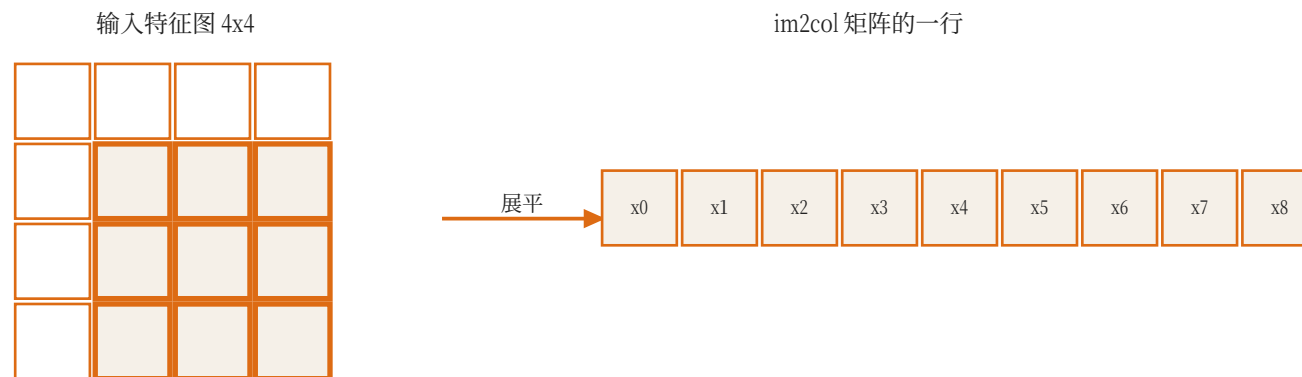


图 10：每个输出像素的 3x3x通道 窗口展开成一行，
整个卷积就变成矩阵乘。

Paper: Sze et al., Efficient Processing of Deep Neural Networks, Proc. IEEE, 2017

Chapter 3 | Line Buffer

Line Buffer 是另一种方案：逐行扫描图片时，只保留最近 3 行，窗口在 buffer 里滑动。这样每个像素只从外部读一次，带宽更省，但地址/时序逻辑更复杂。

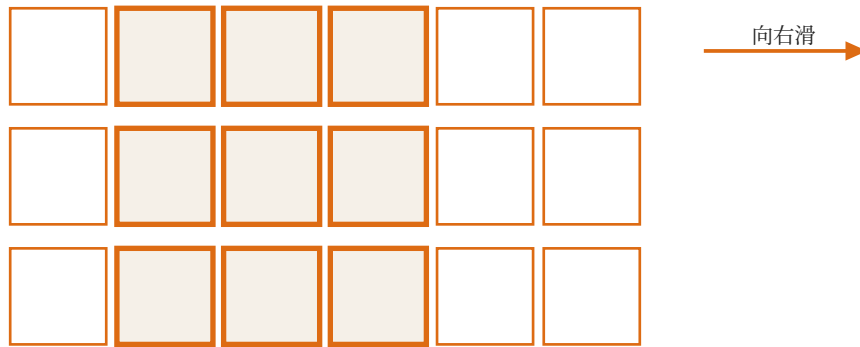
两种方案没有绝对优劣：im2col 简单直接，line buffer 更适合流式/低带宽设计。本章先实现一种即可。

Line Buffer 的思路是“边看边扔”：读图片时只保留最近 3 行，窗口在内存里滑动，滑过去不再需要的行就扔掉。

它比 im2col 省内存和带宽，但地址计算更绕。两种方案都可以，第一次做建议先选 im2col，跑通后再优化。

- 逐行扫描图片时，只需要保留最近 3 行。
- line buffer 可以避免整张图重复搬运。
- 适合流式卷积，减少 SRAM 带宽。
- 两种方案都可以，本章不强制。

图 12：只保留最近 3 行，窗口逐列滑动，避免整图重复搬运。



Chapter 3 | ReLU 与量化

阵列输出是 int32 累加和，需要先乘上 scale 反量化，再重新量化成 int8 给下一层。ReLU 在量化域里只是 $\max(x, 0)$ ，非常便宜。

很多模型用 LeakyReLU/SiLU，硬件实现更贵（需要乘法或查表）。我们的 tiny CNN 先选 ReLU，降低 RTL 难度。

ReLU 的中文意思就是“负数清零”： z 小于 0 就输出 0，否则原样输出。它像把成绩单里的负分全部改成 0。

量化域里做 ReLU 更简单：int8 的负数直接变 0，不需要查表、不需要浮点。这也是为什么小模型优先选 ReLU。

- int32 累加结果需要反量化并重新量化到 int8。
- ReLU 在量化域里就是 $\max(x, 0)$ 。
- LeakyReLU 需要查表或移位近似。
- 本章先用普通 ReLU。

Chapter 3 | MaxPool

MaxPool 2x2 把每 2x2 窗口取最大值，宽高减半。它不做乘法，只用一个比较器和一个计数器，跟在卷积输出后面流式完成。

池化让特征图变小，也减少下一层计算量；对小模型来说这是不可或缺的降采样手段。

MaxPool 2x2 把图片分成很多 2x2 的小方块，每个方块只留最大值。相当于把 4 张图缩成 1 张，信息量减少但保留最明显的特征。

实现上只需要比较器：两个数比大小，大的留下。它不占用乘法器，是 NPU 里最便宜的层之一。

- 2x2 窗口取最大值，stride=2 时宽高减半。
- 可以跟在卷积输出后做流式处理。
- 实现简单：比较器 + 计数器。
- Pooling 不做乘法，不占用阵列。

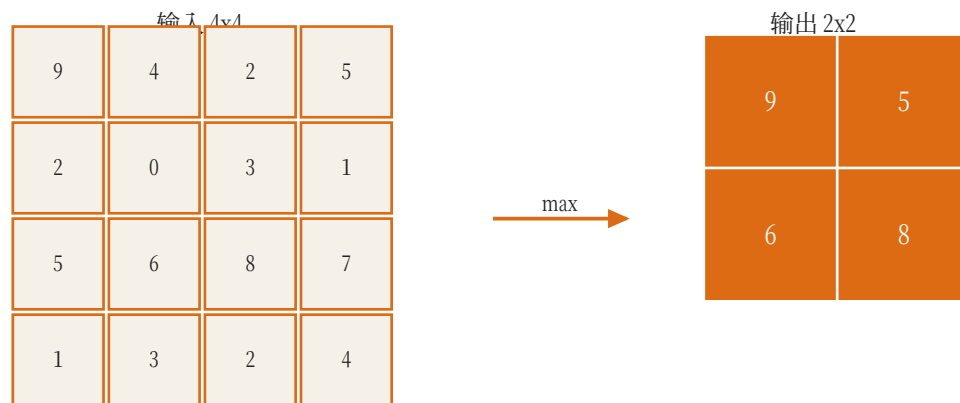


图 13：2x2 窗口取最大值，尺寸减半。

Chapter 3 | 全连接 = GEMM

全连接层 $y = Wx + b$ 就是矩阵乘。输入 784 维、输出 64 维，权重是 64×784 的矩阵。控制器不需要滑动窗口，直接把输入向量按 8 通道一组喂给阵列即可。

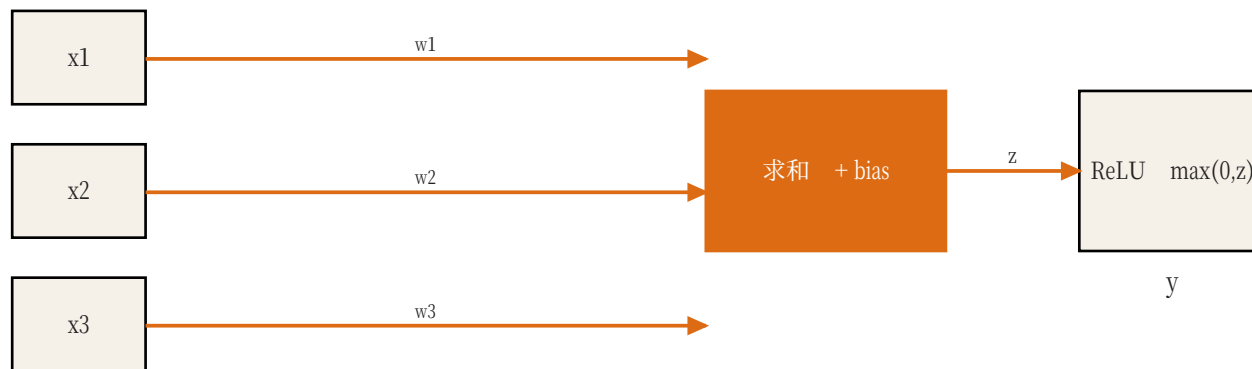
这意味着同一个阵列、同一套控制逻辑，既能做卷积也能做 FC，NPU 的复用性就在这里。

全连接层就像点名册：784 个学生（输入特征）每人乘一个权重，加总后得到 64 个“班级总分”（输出特征）。所有学生 x 所有权重就是一张大矩阵乘法表。

NPU 不需要为 FC 另造硬件：卷积和 FC 都喂给同一个阵列，只是控制器的“送数据方式”不同。

- FC 层就是矩阵乘：输入向量 x 权重矩阵。
- 复用同一个控制器，只是没有滑动窗口。
- MNIST 小模型最后的 $784 \rightarrow 64 \rightarrow 10$ 都是 FC。
- 阵列位宽不足时按 8 通道 tiling。

图 D：神经元把输入分别乘以权重、加总、再加偏置，最后过一个激活函数。



Chapter 3 | Golden 验证

Golden 是“标准答案”：先用 Python 按 int8 规则逐层算出结果，导出成 hex；RTL testbench 读入同样的权重和输入，逐元素比对。

关键是 Python 参考实现必须和 RTL 使用完全相同的量化规则（scale、round、clamp），否则两边永远对不上。

Golden 就是“标准答案”。就像考试前老师先做一遍卷子，把答案锁在保险柜里，学生（RTL）做完后拿出来对。

关键是老师（Python）和学生（RTL）必须用同一套规则：同样的取整、同样的截断、同样的数据顺序。规则不一致，两个人都会觉得自己是对的。

- 用 Python 写 int8 参考实现。
- 每层输出导出为 hex 文件。
- RTL testbench 逐层比对。
- 先单层验证，再做层间串联。

Chapter 3 | 本章任务

- 实现卷积控制器与数据通路。
- 实现 ReLU 与 MaxPool。
- 实现 FC 层。
- 与 Python golden 逐元素一致。

Chapter 3 | 总结

- 控制器决定“什么时候把什么数据送到阵列”。
- im2col 与 line buffer 是两种空间换带宽的方案。
- 激活/池化跟在累加之后，仍是数据流的一部分。
- 下一章解决“权重从哪来”：模型训练与量化导出。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。

References / 延伸阅读

- Sze et al., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey", Proceedings of the IEEE, 2017.
- Chen et al., "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks", ISSCC 2016.
- Han et al., "EIE: Efficient Inference Engine on Compressed Deep Neural Network", ISCA 2016.
- Han et al., "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding", ICLR 2016.